GPU Programming with CUDA and Triton

From Zero to 2-Hours Hero

March 27, 2026

Workshop Outline

Why GPU Programming?

CUDA: Vector Addition

CUDA: Matrix Multiply

Introduction to Triton

Triton in Practice

Wrap-Up

Bonus: Flash Attention

Workshop Roadmap

Part 1: Why GPUs?
CPU vs GPU, memory hierarchy

Part 2: CUDA – Vector Addition
Grid, blocks, threads → code

Part 3: CUDA – Matrix Multiply
Shared memory, bank conflicts, tiling

Part 4: Introduction to Triton
Block-level programming model

Part 5: Triton in Practice
Softmax, matmul, auto-tuning

Part 6: Wrap-Up
Flash Attention (bonus, if time)

Hands-on exercises

Workshop repo: `github.com/ayghri/gpuwork`
GPU playground: `leetgpu.com` – paste code, run, view PTX
Compiler explorer: `godbolt.org` – CUDA → PTX → SASS

CPU vs GPU: Design Philosophy

CPU – Latency-optimized

- Few powerful cores (8–64)
- Large caches, branch prediction
- Optimized for *sequential* tasks
- Low latency per operation

GPU – Throughput-optimized

- Thousands of simple cores
- Small caches, no branch prediction
- Optimized for *parallel* tasks
- High throughput overall

Analogy

CPU = sports car (fast, one passenger)
GPU = bus (slower per trip, but moves 100 people at once)

GPU Architecture Overview

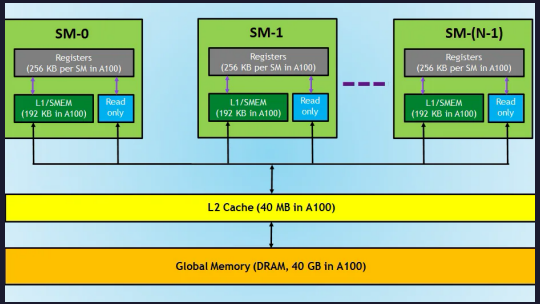
NVIDIA GPU hierarchy:

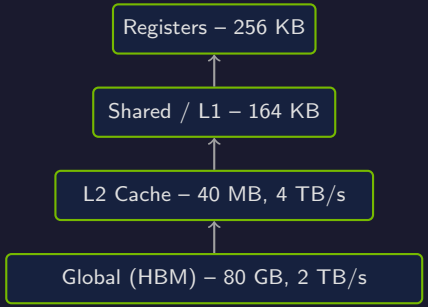
- **GPU** has many Streaming Multiprocessors (SMs)
- Each **SM** has:
  - CUDA cores (INT32, FP32, FP64)
  - Tensor Cores (matrix ops)
  - Shared memory / L1 cache
  - Register file
  - Warp schedulers
- A **warp** = 32 threads executing in lockstep (SIMT)

Example: A100

- 108 SMs
- 6912 FP32 cores
- 432 Tensor Cores
- 80 GB HBM2e
- 2 TB/s bandwidth
- 1.41 GHz boost clock

GPU Memory Hierarchy





The key to GPU performance: **keep data close to the compute**.

The Memory Wall

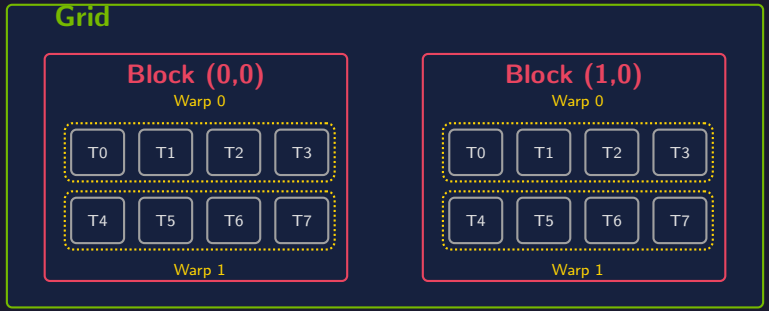
- Most GPU kernels are memory-bound, not compute-bound
- A100: 312 TFLOPS (FP16) but only 2 TB/s memory bandwidth
- Arithmetic intensity = FLOPs / Bytes loaded

Example: Element-wise ReLU

- Load 1 float (4 bytes), 1 comparison, 1 store (4 bytes)
- Arithmetic intensity = 1/8 = 0.125 FLOP/byte
- Completely memory-bound – GPU compute units are idle most of the time

This is why **kernel fusion** matters: combine multiple operations into one kernel to reduce memory round-trips.

Grid, Blocks, and Threads



- **Thread**: single execution unit, has a unique ID
- **Warp**: group of threads executing in lockstep (32 in reality, 4 here)
- **Block (CTA)**: group of threads that can cooperate (shared memory, sync)
- **Grid**: collection of blocks that execute a kernel

Thread Indexing

Every thread knows *who it is* via built-in variables:

Built-in variables

`threadIdx.x, .y, .z`
Thread index within block

`blockIdx.x, .y, .z`
Block index within grid

`blockDim.x, .y, .z`
Threads per block

`gridDim.x, .y, .z`
Blocks per grid

Global thread ID (1D)

```
int i = blockIdx.x * blockDim.x
      + threadIdx.x;
```

With 4 blocks of 256 threads:
Block 0: threads 0–255
Block 1: threads 256–511
Block 2: threads 512–767
Block 3: threads 768–1023

CPU Baseline: Vector Addition

Before writing GPU code, let's see the CPU version:

Sequential

```
void vec_add(const float* A, const float* B, float* C, int N) {
    for (int i = 0; i < N; i++)
        C[i] = A[i] + B[i];
}
```

Parallel with OpenMP

```
void vec_add_omp(const float* A, const float* B, float* C, int N) {
    #pragma omp parallel for
    for (int i = 0; i < N; i++)
        C[i] = A[i] + B[i];
}
```

- OpenMP parallelizes across CPU cores (8–64 threads)
- GPU will launch **thousands** of threads for the same work
- File: `00_cpu_vec_add.cu`

First Kernel: Vector Addition

Kernel – runs on GPU, one instance per thread

```
__global__ void vec_add(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}
```

Line by line

- `__global__` – marks this as a **kernel**: called from CPU, runs on GPU
- `blockIdx.x * blockDim.x + threadIdx.x` – each thread computes its unique global index
- `if (i < n)` – boundary guard: grid may have more threads than elements
- `c[i] = a[i] + b[i]` – the actual work: each thread handles **one** element

What Does the GPU Actually Run? PTX

`nvcc` compiles CUDA to **PTX** (Parallel Thread Execution) – a virtual ISA for NVIDIA GPUs.

PTX for c[i] = a[i] + b[i] (simplified)

```
ld.global.f32  %f1, [%rd4];   // load a[i]
ld.global.f32  %f2, [%rd5];   // load b[i]
add.f32        %f3, %f1, %f2; // f3 = a[i] + b[i]
st.global.f32  [%rd6], %f3;   // store c[i]
```

- PTX is then compiled to **SASS** (the actual machine code) for your specific GPU
- `nvcc -ptx vec_add.cu` generates the PTX file
- LeetGPU shows the generated PTX for your submissions
- `godbolt.org` lets you view CUDA → PTX → SASS interactively

Why care about PTX?

Reading PTX tells you what the compiler *actually* did – did it use shared memory? How many registers? Did it unroll your loop?

Exercise 1c: Vec Add with Inline PTX

File: `01c_ptx_vec_add.cu` (nvcc only – use godbolt.org or compile locally)

You can embed PTX instructions directly in CUDA using `asm()`.

```
__global__ void vec_add_ptx(const float* A, const float* B,
                            float* C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        float a = A[i], b = B[i], c;
        asm("add.f32 %0, %1, %2;" : "=f"(c) : "f"(a), "f"(b));
        C[i] = c;
    }
}
```

- `"add.f32 %0, %1, %2"` – the PTX instruction
- `"=f"(c)` – output: float register mapped to c
- `"f"(a), "f"(b)` – inputs: float registers mapped to a, b

Thread Identity: CUDA Built-ins and PTX Registers

Every thread knows who it is. Some identities are CUDA built-ins, others require PTX:

Works on LeetGPU

```
unsigned int laneid = threadIdx.x % 32;  // warp lane
unsigned int warpid = threadIdx.x / 32;  // warp index
```

CUDA built-in	PTX register	Meaning
`threadIdx.x/y/z`	`%tid.x/y/z`	Thread index in block
`blockDim.x/y/z`	`%ntid.x/y/z`	Block dimensions
`blockIdx.x/y/z`	`%ctaid.x/y/z`	Block index in grid
`gridDim.x/y/z`	`%nctaid.x/y/z`	Grid dimensions
`threadIdx.x % 32`	`%laneid`	Lane within warp (0–31)
`threadIdx.x / 32`	`%warpid`	Warp index within block
`clock()`	`%clock`	Cycle counter
(inline PTX only)	`%smid`	Which SM runs this block

- File: `01d_special_registers.cu` – prints warp, lane, block for each thread

Host Code: Allocate, Copy, Launch, Copy Back

```
int n = 1 << 20;  // 1M elements
size_t bytes = n * sizeof(float);

// Allocate host (CPU) and device (GPU) memory
float *h_a = (float*)malloc(bytes);     // CPU
float *d_a; cudaMalloc(&d_a, bytes);    // GPU

// Copy data: CPU -> GPU
cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);

// Launch kernel: 4096 blocks x 256 threads = 1M threads
vec_add<<<(n+255)/256, 256>>>(d_a, d_b, d_c, n);

// Copy result: GPU -> CPU
cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);

// Free GPU memory
cudaFree(d_a);
```

`<<<grid_size, block_size>>>` = how many blocks, how many threads per block

What Just Happened?

- CPU and GPU have **separate memory** – data must be copied explicitly
- The kernel launches **asynchronously** – CPU doesn't wait
- `cudaMemcpy` back to host **synchronizes** (waits for kernel to finish)
- For vec_add with 1M elements: 4096 blocks $\times$ 256 threads $=$ 1,048,576 threads launched simultaneously

Warps and SIMT

- Threads in a block are grouped into **warps** of 32 threads
- All threads in a warp execute the *same instruction* at the same time (**SIMT**)
- If threads in a warp diverge (different `if` branches):
  $\rightarrow$ both branches execute sequentially (warp divergence)

Warp Divergence – Avoid This

```
if (threadIdx.x % 2 == 0) { do_A(); }
else { do_B(); }
```

Half the warp is idle during `do_A()`, the other half during `do_B()`. 50% efficiency!

In vec_add, the `if (i < n)` guard only diverges in the *last* block – negligible cost.

Exercise 1: Vector Addition

Your turn! Open `leetgpu.com` and try it

File: `01_vec_add.cu`

```
__global__ void vec_add_kernel(const float* A, const float* B,
                               float* C, int N) {
    int i = 0; // TODO: compute global thread index

    // TODO: boundary check and perform addition
}
```

- Compute `i` using `blockIdx.x`, `blockDim.x`, `threadIdx.x`
- Don't forget the `if (i < N)` guard!
- After it passes, check the PTX tab on LeetGPU – find the `ld.global`, `add.f32`, `st.global`

Exercise 1b: Fused Vector Add + ReLU

Try it on `leetgpu.com`

File: `01b_fused_vec_add_relu.cu`

Remember the **memory wall**? Two separate kernels means two HBM round-trips:

```
// UNFUSED: 4 HBM accesses per element
vec_add_kernel<<<G,B>>>(A, B, tmp, N);  // read A,B -> write tmp
relu_kernel<<<G,B>>>(tmp, C, N);        // read tmp -> write C
```

Fuse them into **one kernel** – only 2 HBM accesses per element:

```
__global__ void fused_vec_add_relu_kernel(const float* A,
        const float* B, float* C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        C[i] = 0.0f; // TODO: fmaxf(A[i] + B[i], 0.0f)
    }
}
```

Key takeaway: kernel fusion = fewer memory round-trips = faster.

Naive Matrix Multiplication

```
__global__ void matmul_naive(float *A, float *B, float *C,
                             int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < K; k++) {
            sum += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

- Each thread computes one element of C – now using **2D** blocks/grid
- Launch: `dim3 block(16, 16); dim3 grid(N/16, M/16);`
- Problem: each element of A and B is loaded from global memory K times!

Exercise 2: Naive Matrix Multiplication

Try it on leetgpu.com

File: 02_naive_matmul.cu

```
__global__ void matmul_kernel(const float* A, const float* B,
                              float* C, int M, int N, int K) {
    int row = 0; // TODO: use blockIdx.y, blockDim.y, threadIdx.y
    int col = 0; // TODO: use blockIdx.x, blockDim.x, threadIdx.x

    if (row < M && col < N) {
        float sum = 0.0f;
        // TODO: loop over K, accumulate A[row][k] * B[k][col]
        C[row * N + col] = sum;
    }
}
```

- Row-major layout: `A[row][k]` = `A[row * K + k]`
- Launch with `dim3 block(16, 16)` – 256 threads per block

Why the Naive Version is Slow

For $C = A \times B$ with $M = N = K = 4096$:

- Each thread does $K = 4096$ multiply-adds, loading 2 floats each time
- 16.8M threads $\times$ 4096 $\times$ 2 $\times$ 4 bytes = **549 GB** of global loads
- But row i of A is needed by every element in row i of C (4096 times!)
- Same for column j of B – massive redundant traffic

We need to cache and reuse data

Global memory: $\sim$400 cycles per access.
What if we loaded a tile once into **fast on-chip memory** and reused it?

Shared Memory: On-Chip Cache You Control

Fast memory shared by all threads in a block – a programmer-managed SRAM.

Declaration and usage

```
__shared__ float tile[16][16]; // per-block, on-chip

// Each thread loads one element from global -> shared
tile[threadIdx.y][threadIdx.x] = A[row * K + col];
__syncthreads(); // barrier: everyone must finish loading

// Now any thread in the block can read any element -- fast
float val = tile[threadIdx.y][3]; // ~20 cycles vs ~400
```

- **48–164 KB per SM** (configurable, shared with L1)
- ∼20 cycles latency vs ∼400 for global memory
- `__syncthreads()`: barrier – ensures all threads finished writing before any reads

Tiled Matmul: Loading Tiles into Shared Memory

```
#define TILE 16
__global__ void matmul_tiled(float *A, float *B, float *C,
                             int M, int N, int K) {
    __shared__ float As[TILE][TILE], Bs[TILE][TILE];
    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < (K + TILE - 1) / TILE; t++) {
        // All 256 threads collaboratively load 2 tiles
        As[threadIdx.y][threadIdx.x] =
            A[row * K + t * TILE + threadIdx.x];
        Bs[threadIdx.y][threadIdx.x] =
            B[(t * TILE + threadIdx.y) * N + col];
        __syncthreads();

        for (int k = 0; k < TILE; k++)
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }
    C[row * N + col] = sum;
}
```

Cache Reuse: Why Tiling Works

Without tiling

Every multiply-add loads from HBM.

Global loads: $O(MNK)$
$= 2 \times 4096^3 \times 4 \approx$ **549 GB**

With tiling ($T = 16$)

Each tile loaded once, reused T times.

Global loads: $O(MNK/T)$
$\approx$ **34 GB** – $16\times$ less!

What happens inside one tile iteration:

1. 256 threads each load 1 element of A tile + 1 element of B tile from global $\rightarrow$ shared
2. `__syncthreads()` – everyone waits
3. Each thread reads 16 elements from A_s and 16 from B_s –**all from shared memory**
4. `__syncthreads()` – safe to overwrite for next tile

Shared Memory Banks

Shared memory is divided into **32 banks** (one per warp thread).
Consecutive 4-byte words map to consecutive banks: (example with 8 banks in SMEM)

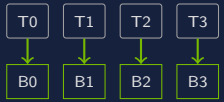
Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
addr 00	addr 01	addr 02	addr 03	addr 04	addr 05	addr 06	addr 07
addr 08	addr 09	addr 10	addr 11	addr 12	addr 13	addr 14	addr 15
addr 16	addr 17	addr 18	addr 19	addr 20	addr 21	addr 22	addr 23

...

- Each bank serves one address per cycle
- Two threads, *different addresses*, *same bank*: **bank conflict** – serialized
- All threads, *same address*: broadcast (no conflict)

Bank Conflicts in Our Matmul

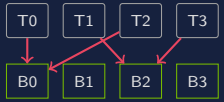
`As[ty][k]` – no conflict



All threads in a warp read the same `k`, different `ty` → different rows → different banks.

No conflict

`Bs[k][tx]` – depends on `k`



If row stride is a multiple of 32, threads reading different columns can hit the same bank.

Potential conflict!

Fixing Bank Conflicts: Padding

The trick: add one extra column

```
__shared__ float Bs[TILE][TILE];          ← stride 16: bank conflicts on column access
__shared__ float Bs[TILE][TILE + 1];      ← stride 17: no conflicts
```

The padding shifts each row by one bank, breaking the conflict pattern.
Cost: 16 extra floats (64 bytes) of shared memory per tile – negligible.

Other strategies

- Rearrange access patterns so threads hit consecutive banks
- Use warp-level shuffles (`__shfl_sync`) to exchange data without shared memory
- Profile with `ncu` – it reports bank conflicts directly

Exercise 3: Tiled Matmul with Shared Memory

Try it on leetgpu.com

File: 03_tiled_matmul.cu

```
// Inside the tile loop:

// TODO: load one element each into shared memory
As[threadIdx.y][threadIdx.x] = A[???];
Bs[threadIdx.y][threadIdx.x] = B[???];
__syncthreads();

// TODO: accumulate from shared memory
for (int k = 0; k < TILE; k++)
    sum += As[???][???] * Bs[???][???];
__syncthreads();
```

- Tile t starts at column `t*TILE` for A, row `t*TILE` for B
- Add boundary checks (load `0.0f` if out of bounds)
- Compare performance with your Exercise 2 solution, look for `ld.global`

Exercise 4: Fix Bank Conflicts

Try it on leetgpu.com

File: 04_bank_conflict_fix.cu

Take your working tiled matmul from Exercise 3 and:

1. Run it as-is and note the performance
2. Change the Bs declaration from:

```
__shared__ float Bs[TILE][TILE];        // bank conflicts
__shared__ float Bs[TILE][TILE + 1];    // no conflicts
```

3. Run again – do you see a speedup?

What to expect

The improvement depends on matrix size and GPU.
On large matrices, padding can give 5–15% speedup.

CUDA Optimization Summary

1. **Coalesce global memory**: consecutive threads $\rightarrow$ consecutive addresses
2. **Cache in shared memory**: load tiles, reuse across threads
3. **Avoid bank conflicts**: pad shared memory arrays
4. **Minimize warp divergence**: avoid branching within warps
5. **Maximize occupancy**: enough blocks/threads to keep SMs busy

Profiling tools

`nsys` (Nsight Systems) – timeline view
`ncu` (Nsight Compute) – kernel-level metrics (bank conflicts, occupancy, . . .)

What About Tensor Cores?

Our tiled matmul uses **FP32 CUDA cores** – one multiply-add per core per cycle.
Modern GPUs have **Tensor Cores**: dedicated matrix-multiply units.

Tensor Core MMA (Matrix Multiply-Accumulate)

- Computes a small matrix multiply (e.g. $16 \times 16 \times 16$) in **one operation**
- A100: **312 TFLOPS** (FP16) vs 19.5 TFLOPS (FP32 CUDA cores) – $16\times$ faster
- Requires specific data layouts in registers across threads in a warp

The catch: thread-level data layout

Each of the 32 warp threads holds a *fragment* of the tile in its registers.
The layout depends on the GPU architecture (Volta, Ampere, Hopper...).
Getting this right in CUDA requires `wmma` or `mma.sync` PTX intrinsics – complex!

From CUDA Pain to Triton

Writing high-performance CUDA is hard:

- Manual memory management, tiling, synchronization
- Bank conflict avoidance, memory coalescing
- Tensor Core usage requires warp-level register fragment layouts
- Different optimizations per GPU architecture
- Hundreds of lines for a "simple" matmul

What if the compiler handled all of that?

Triton's promise

You write block-level Python. The compiler generates:
shared memory staging, bank-conflict-free layouts, coalesced loads,
and **Tensor Core instructions** – automatically.

What is Triton?

- Open-source language and compiler by OpenAI (2021)
- Write GPU kernels in **Python** with near-CUDA performance
- Key idea: program at the **block level**, not the thread level
- The compiler handles:
  - Shared memory management
  - Memory coalescing
  - Warp-level optimizations
  - Tensor Core utilization

Why Triton?

- $\sim$10$\times$ less code than equivalent CUDA
- 90–100% of hand-tuned CUDA performance
- Python-native: integrates with PyTorch seamlessly
- Active development, used in production (PyTorch 2.0 `torch.compile`)

CUDA vs Triton: Programming Model

CUDA: Thread-level

- You write code for *one thread*
- You manage:
  - Thread indexing
  - Shared memory
  - Synchronization
  - Memory coalescing
- Full control, full responsibility

Triton: Block-level

- You write code for *one block*
- Compiler manages:
  - Thread mapping
  - Shared memory
  - Synchronization
  - Memory coalescing
- Less control, less boilerplate

In Triton, you think in **tiles/blocks of data**, not individual elements.

Triton Core Concepts

- **Program**: a Triton kernel – one instance per block
- `tl.program_id(axis)`: like `blockIdx` in CUDA
- `tl.arange(0, N)`: creates a vector `[0, 1, ..., N-1]` within a block
- `tl.load(ptr)`: loads a block of data from global memory
- `tl.store(ptr, val)`: stores a block of data to global memory
- **Masking**: `tl.load(ptr, mask=mask, other=0.0)` for boundary handling

Key difference from NumPy/PyTorch

Operations in Triton are **block-local**: each kernel instance operates on its own tile. There is no implicit broadcasting across the entire tensor.

Triton: Vector Addition

```python
import triton
import triton.language as tl

@triton.jit
def vec_add_kernel(a_ptr, b_ptr, c_ptr, n,
                   BLOCK: tl.constexpr):
    pid = tl.program_id(0)
    offsets = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n

    a = tl.load(a_ptr + offsets, mask=mask)
    b = tl.load(b_ptr + offsets, mask=mask)
    tl.store(c_ptr + offsets, a + b, mask=mask)
```

Launching the kernel

```python
grid = lambda meta: (triton.cdiv(n, meta['BLOCK']),)
vec_add_kernel[grid](a, b, c, n, BLOCK=1024)
```

Compare: 6 lines of kernel code vs 25+ lines in CUDA (with host code).

Exercise 5: Triton Vector Addition

Run locally (requires GPU + `pip install triton torch`)

File: `05_triton_vec_add.py`

- The file is a complete, working Triton vec_add – run it and read through the code
- Compare with your CUDA Exercise 1:
  - No `cudaMalloc`, no `cudaMemcpy` – PyTorch handles memory
  - `tl.program_id(0)` ≡ `blockIdx.x`
  - `tl.arange(0, BLOCK)` creates a vector of indices – no manual thread indexing
  - `mask = offsets < N` replaces the `if (i < N)` guard
- Try changing `BLOCK=1024` to other values – how does it affect performance?

Triton: Fused Softmax

Computing $\text{softmax}(x)_i = e^{x_i - \max(x)} / \sum_j e^{x_j - \max(x)}$

In PyTorch: 5 global memory round-trips (max, subtract, exp, sum, divide).
In Triton: **one kernel**, data stays in SRAM.

```python
@triton.jit
def softmax_kernel(input_ptr, output_ptr, n_cols,
                   BLOCK: tl.constexpr):
    row = tl.program_id(0)
    offsets = tl.arange(0, BLOCK)
    mask = offsets < n_cols
    # Load one row
    row_ptr = input_ptr + row * n_cols
    x = tl.load(row_ptr + offsets, mask=mask,
                other=-float('inf'))
    # Compute softmax in SRAM
    x_max = tl.max(x, axis=0)
    numerator = tl.exp(x - x_max)
    denominator = tl.sum(numerator, axis=0)
    result = numerator / denominator
    out_ptr = output_ptr + row * n_cols
    tl.store(out_ptr + offsets, result, mask=mask)
```

Why Fused Softmax is Faster

PyTorch (unfused)

1. Read X, write $\max(X)$
2. Read $X, \max(X)$, write $X - \max$
3. Read $X - \max$, write $\exp(X - \max)$
4. Read exp, write $\sum \exp$
5. Read exp, $\sum$, write result

5 kernel launches, 5 global memory round-trips

Triton (fused)

1. Read X into SRAM
2. Compute everything in SRAM
3. Write result

1 kernel launch, 1 read + 1 write

Typical speedup: **2–4**$\times$ over PyTorch for memory-bound ops.

Exercise 6: Triton Fused Softmax

Run locally

File: `06_triton_softmax.py`

- Run the file – it computes row-wise softmax and checks against `torch.softmax`
- Read the kernel: one `tl.load`, all compute in SRAM, one `tl.store`
- Try these experiments:
  - Change the matrix to `(128, 8192)` – does it still work? (hint: `BLOCK` must be $\geq$ `n_cols`)
  - What happens if you remove `other=-float("inf")` from the load?
  - Time it vs `torch.softmax` using `torch.cuda.Event` for large matrices

Triton: Matrix Multiplication (1/2)

```python
@triton.jit
def matmul_kernel(
    A, B, C, M, N, K,
    stride_am, stride_ak, stride_bk, stride_bn,
    stride_cm, stride_cn,
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr,
    BLOCK_K: tl.constexpr,
):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    # Block starting row/col
    rm = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    rn = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    rk = tl.arange(0, BLOCK_K)

    # Pointers to first tiles of A and B
    A_ptrs = A + rm[:, None] * stride_am + rk[None, :] * stride_ak
    B_ptrs = B + rk[:, None] * stride_bk + rn[None, :] * stride_bn
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
```

Triton: Matrix Multiplication (2/2)

```
    # Tiled loop over K dimension
    for k in range(0, K, BLOCK_K):
        a = tl.load(A_ptrs, mask=
            (rm[:, None] < M) & (rk[None, :] + k < K),
            other=0.0)
        b = tl.load(B_ptrs, mask=
            (rk[:, None] + k < K) & (rn[None, :] < N),
            other=0.0)
        acc += tl.dot(a, b)
        A_ptrs += BLOCK_K * stride_ak
        B_ptrs += BLOCK_K * stride_bk

    # Store result
    C_ptrs = C + rm[:, None] * stride_cm + rn[None, :] * stride_cn
    mask = (rm[:, None] < M) & (rn[None, :] < N)
    tl.store(C_ptrs, acc, mask=mask)
```

- `tl.dot`: compiles to Tensor Core instructions (when available)
- Shared memory, tiling, coalescing: all handled by the compiler
- $\sim$30 lines vs $\sim$150+ lines for an optimized CUDA matmul

Auto-Tuning in Triton

Triton can automatically search for the best kernel configuration:

```
@triton.autotune(
    configs=[
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 128,
                       'BLOCK_K': 32}, num_warps=8),
        triton.Config({'BLOCK_M': 64, 'BLOCK_N': 256,
                       'BLOCK_K': 32}, num_warps=8),
        triton.Config({'BLOCK_M': 256, 'BLOCK_N': 64,
                       'BLOCK_K': 32}, num_warps=4),
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 64,
                       'BLOCK_K': 64}, num_warps=4),
    ],
    key=['M', 'N', 'K'],  # re-tune when shapes change
)
@triton.jit
def matmul_kernel(A, B, C, M, N, K, ...):
    ...
```

Triton benchmarks all configs on first call, caches the winner.

Exercise 7: Triton Matrix Multiplication

Run locally

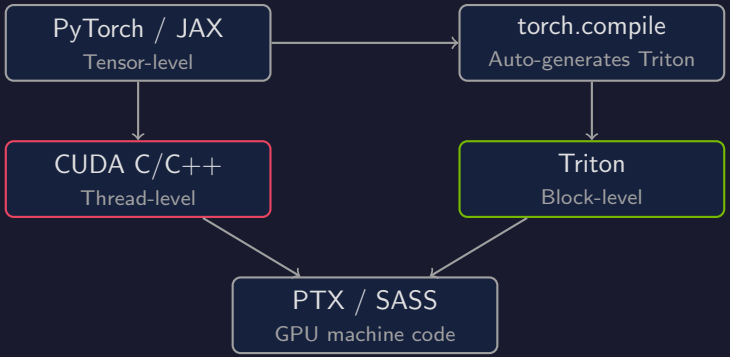
File: `07_triton_matmul.py`

- Run the file – it multiplies two 512×512 matrices and checks against `torch.mm`
- Compare the code with your CUDA tiled matmul (Exercise 3):
  - No `__shared__`, no `__syncthreads()` – compiler handles it
  - `tl.dot(a, b)` compiles to Tensor Core instructions automatically
  - Boundary masking via `mask=` instead of manual `if` checks
- Try experiments:
  - Increase to 2048×2048 – how does Triton compare to `torch.mm` (cuBLAS)?
  - Change `BLOCK_M/N/K` values – what gives best performance?

CUDA vs Triton: When to Use What?

	CUDA	Triton
Learning curve	Steep	Gentle
Code length	Verbose	Concise
Peak performance	Highest	Near-CUDA
Thread-level control	Full	Limited
Shared memory control	Manual	Automatic
Auto-tuning	Manual/external	Built-in
Warp-level primitives	Yes	Limited
Non-NVIDIA GPUs	No	Experimental (AMD)
PyTorch integration	Requires C++ bindings	Native
Best for	Libraries, max perf	Custom ops, research

The GPU Programming Landscape



`torch.compile` (PyTorch 2.0+) auto-generates Triton kernels from eager PyTorch code – often good enough!

Summary

1. **GPUs** are throughput machines – thousands of cores, memory hierarchy is key
2. **CUDA** gives full control: grid → blocks → warps → threads
   - Shared memory for cache reuse, but you manage tiling, sync, bank conflicts
   - Tensor Cores are fast but require complex warp-level layouts
3. **Triton** raises the abstraction: block-level programming in Python
   - Compiler handles shared memory, coalescing, Tensor Cores
   - Built-in auto-tuning, native PyTorch integration
4. **Kernel fusion** is the single biggest optimization for most workloads

Resources

Workshop files: `github.com/ayghri/gpuwork`
Triton: `triton-lang.org` — CUDA: `docs.nvidia.com/cuda`
PTX ISA: `docs.nvidia.com/cuda/parallel-thread-execution`
Compiler Explorer: `godbolt.org` — GPU MODE: YouTube / Discord

Questions?

Bonus Material

If time allows: Flash Attention

Flash Attention: The Idea

Standard attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$

Standard implementation

1. Compute $S = QK^\top$ – $O(n^2 d)$
2. Store full $n \times n$ matrix S
3. Compute $\text{softmax}(S)$ row-wise
4. Compute $\text{softmax}(S) \cdot V$

Memory: $O(n^2)$ – explodes for long sequences

Flash Attention

1. Tile Q, K, V into blocks
2. For each block of Q: iterate over K, V blocks
3. Compute local softmax with online correction
4. Never materialize full $n \times n$ matrix

Memory: $O(n)$ – fits in SRAM!

Result: 2–4× faster, uses ∼10–20× less memory.

Online Softmax: The Key Trick

How to compute softmax without seeing all values at once?

Process blocks $B_1, B_2, \ldots$ incrementally:

1. After block B_1: compute $m_1 = \max(B_1)$, $\ell_1 = \sum e^{B_1 - m_1}$
2. After block B_2: update running max $m_2 = \max(m_1, \max(B_2))$
3. **Rescale** previous result: $\ell_1' = \ell_1 \cdot e^{m_1 - m_2}$
4. Accumulate: $\ell_2 = \ell_1' + \sum e^{B_2 - m_2}$

Applied to attention

- Maintain running output O, running max m, running sum ℓ
- For each new K, V block: rescale O by the max correction factor
- Mathematically exact – not an approximation!

Flash Attention in Triton (Sketch)

```python
@triton.jit
def flash_attn_kernel(Q, K, V, Out, ...):
    # This program handles one block of Q rows
    q_block = tl.load(Q_block_ptr)  # [BLOCK_M, d]
    m_i = tl.full([BLOCK_M], -float('inf'), tl.float32)
    l_i = tl.zeros([BLOCK_M], dtype=tl.float32)
    acc = tl.zeros([BLOCK_M, D], dtype=tl.float32)

    for j in range(0, N, BLOCK_N):  # iterate K/V blocks
        k_block = tl.load(K_block_ptr)  # [BLOCK_N, d]
        v_block = tl.load(V_block_ptr)  # [BLOCK_N, d]
        # Compute QK^T for this block
        s = tl.dot(q_block, tl.trans(k_block)) / sqrt_d
        # Online softmax update
        m_new = tl.maximum(m_i, tl.max(s, axis=1))
        alpha = tl.exp(m_i - m_new)
        p = tl.exp(s - m_new[:, None])
        l_i = l_i * alpha + tl.sum(p, axis=1)
        acc = acc * alpha[:, None] + tl.dot(p, v_block)
        m_i = m_new
    tl.store(Out_ptr, acc / l_i[:, None])
```